

Levshell

A Context-Aware Research Environment
for the Sway Window Manager

SPECIFICATION DOCUMENT

Version 0.2 — Draft

April 14, 2026

Guiding Principle

A blissful mix of elegance and absolute productivity. A shell that gets out of your way when needed, but is there almost like an omnipotent catalyst to help you through difficult patches of work.

*Built with QuickShell (QML) for rendering and a Rust daemon for logic.
A hybrid architecture: unified data model with incremental tool replacement.
Designed for Wayland/Sway. Engineered for researchers and students.*

Contents

1	Introduction	5
1.1	Hybrid Architecture Strategy	5
1.2	System Components	5
1.3	Design Principles	6
2	Feature Specification	6
2.1	Top Bar Layout & Global UX	6
2.1.1	Global Menu Bar	6
2.1.2	Unified Command Palette	6
2.1.3	Breadcrumb Workspace Indicator	7
2.1.4	Morphing Bar Density	7
2.1.5	Clock & Calendar Hub	7
2.1.6	Notification Center	7
2.1.7	Quick-Settings Flyout	7
2.2	Productivity & Focus	7
2.2.1	Pomodoro / Session Timer	7
2.2.2	Daily Planner Glance	8
2.2.3	Scratch Pad / Quick Capture	8
2.2.4	Reading List Queue	8
2.2.5	Global Word/Character Count	8
2.2.6	Session Journal	8
2.3	System Telemetry & Resource Awareness	8
2.3.1	Adaptive System Telemetry	8
2.3.2	Battery & Power Profile	8
2.3.3	Network Quality Indicator	8
2.3.4	Disk Space Warning	8
2.3.5	Process Sniper	8
2.4	Workspaces & Window Management	9
2.4.1	Named & Persistent Workspaces	9
2.4.2	Workspace Thumbnails	9
2.4.3	Workspace Templates	9
2.4.4	Window Grouping / Tabs	9
2.4.5	Scratchpad Manager	9
2.4.6	Picture-in-Picture	9
2.4.7	Workspace Activity Indicator	9
2.5	File & Clipboard Management	9
2.5.1	Clipboard History	9
2.5.2	Clipboard Transformations	9
2.5.3	Drag-and-Drop Shelf	9
2.5.4	Recent Files	9
2.5.5	Quick File Preview	10
2.6	Search & Knowledge	10
2.6.1	Unified Desktop Search	10
2.6.2	Dictionary / Thesaurus Lookup	10
2.6.3	Quick Translation	10
2.6.4	Formula Reference	10
2.6.5	Snippet / Template Library	10
2.7	Communication & Social	10
2.7.1	Unified Inbox Indicator	10
2.7.2	Quick Reply	10

2.7.3	Meeting Join Button	10
2.7.4	Presence / Status Broadcast	10
2.8	Media & Ambient	11
2.8.1	MPRIS Media Controls	11
2.8.2	Now-Playing Display	11
2.8.3	Per-Application Volume Mixer	11
2.8.4	Ambient Soundscapes	11
2.8.5	Night-Light / Color Temperature	11
2.9	Academic & Research	11
2.9.1	Open Project Registry	11
2.9.2	Stochastic Ideation Nudges	11
2.9.3	Cross-Project Connection Surfacing	11
2.9.4	Project Pulse Indicator	11
2.9.5	Blocked-Project Escalation	12
2.9.6	Anki Integration	12
2.9.7	Obsidian / Knowledge Base Integration	12
2.9.8	Zotero & Literature Management	12
2.9.9	arXiv / Semantic Scholar Watcher	13
2.9.10	L ^A T _E X Compilation Status	13
2.9.11	Lecture / Meeting Recorder	13
2.9.12	Deadline Tracker	13
2.10	SSH & Remote Systems	13
2.10.1	SSH Connection Dashboard	13
2.10.2	Named Host Profiles	13
2.10.3	Remote Job Monitor	13
2.10.4	GPU Utilization Dashboard	13
2.10.5	Remote Filesystem Bookmarks	13
2.10.6	SSH Tunnel Manager	14
2.10.7	Experiment Tracker Integration	14
2.10.8	Cluster Queue Status	14
2.10.9	SSH Session Logging	14
2.11	Code & Development Awareness	14
2.11.1	Build / CI Status	14
2.11.2	Git Status per Project	14
2.11.3	Compiler Error Flash	14
2.11.4	Dependency Vulnerability Alerts	14
2.11.5	Docker / Container Status	14
2.11.6	Lab Notebook	14
2.12	Deep Focus & Cognitive Support	14
2.12.1	Warmup Mode	14
2.12.2	Context Save / Restore	15
2.12.3	Interruption Cost Awareness	15
2.12.4	Literature Review Mode	15
2.12.5	Writing Mode	15
2.12.6	Rubber Duck Debugger	15
2.13	Data & Experiment Pipeline	15
2.13.1	Dataset Registry	15
2.13.2	Storage Quota Monitor	15
2.13.3	Reproducibility Snapshot	15
2.13.4	Paper-to-Code Linker	15
2.14	Health & Wellbeing	15

2.15	Security & Privacy	16
2.16	External Hardware & Peripherals	16
2.17	AI & LLM Integration	16
2.17.1	Inline AI Assistant	16
2.17.2	Smart Paste	16
2.17.3	Contextual Suggestions	16
2.18	Appearance & Theming	16
2.19	Automation & Scripting	16
2.19.1	levshell-ctl: CLI Control Interface	16
2.19.2	User-Defined Widgets	17
2.19.3	Trigger System	17
2.19.4	Keybind Cheat Sheet	17
2.19.5	Shell Command Pinning	17
2.19.6	Extension / Plugin System	17
2.20	Meta-Productivity & Self-Awareness	17
3	Technical Architecture	18
3.1	Core Components	18
3.2	IPC Protocol	19
3.2.1	Transport	19
3.2.2	Codec Abstraction and Serialization Strategy	19
3.2.3	Daemon → QML: State Patches	19
3.2.4	QML → Daemon: User Events	20
3.3	Unified Data Model	20
3.3.1	Core Entity Types	20
3.3.2	Provenance Tracking	22
3.3.3	Cross-Entity Relations	22
3.3.4	Sync Engine	23
3.3.5	Migration Path: Sync Adapter to Native Component	23
3.4	Module System	24
3.4.1	Module Trait	24
3.4.2	Module Isolation	24
3.4.3	Out-of-Process Modules	24
3.4.4	QML Widget Skins	25
3.5	Context Engine	25
3.5.1	Input Signals	25
3.5.2	Resolution Layers	25
3.5.3	Relevance Rule Format	26
3.5.4	Conflict Resolution Algorithm	26
3.5.5	Hysteresis and Flicker Prevention	26
3.6	Event Bus	27
3.6.1	Example Event Types	27
3.7	Project Registry	27
3.8	App-to-Context Mapping	28
3.9	Configuration Architecture	28
3.9.1	Directory Structure	28
3.9.2	Configuration Precedence	29
3.10	Extension Points	29
4	Implementation Decisions	29
4.1	Rust Workspace Layout	29
4.1.1	Crate Dependency Graph	30

4.1.2	Rationale	30
4.2	Database and Schema Decisions	31
4.3	Runtime and Library Decisions	32
4.4	IPC Message Types (Phase 0)	33
4.4.1	Daemon → QML	33
4.4.2	QML → Daemon	33
5	Architectural Constraints & Failure Modes	34
5.1	Integration Brittleness and Sync Adapter Isolation	34
5.1.1	Design Constraint: Modules Never Touch External Tools	34
5.1.2	Sync Adapter Inventory	34
5.1.3	Version Probing	34
5.1.4	Long-Term Brittleness Reduction	35
5.2	Widget Degraded State and Error Handling UI	35
5.2.1	Widget Health States	35
5.2.2	Automatic State Transitions	35
5.2.3	QML-Side Consistency	35
5.3	Power Awareness	36
6	Implementation Roadmap	36
6.1	Phase 0: Foundation	36
6.1.1	Step 0.1: Scaffold the Workspace	36
6.1.2	Step 0.2: Unified Data Model and Store	37
6.1.3	Step 0.3: Event Bus	37
6.1.4	Step 0.4: IPC Server	37
6.1.5	Step 0.5: Daemon Binary and First Module	37
6.2	Phase 1: Core Shell	38
6.2.1	Step 1.1: levshell-ctl CLI	38
6.2.2	Step 1.2: Context Engine	39
6.2.3	Step 1.3: System Telemetry Modules	39
6.2.4	Step 1.4: Core Bar Widgets and Visual Language	40
6.2.5	Step 1.5: Command Palette	40
6.3	Phase 2: Sync Adapters and Research Features	40
6.4	Phase 3: Native Components (Ongoing)	41
6.5	Deferred	41
A	Initial SQLite Schema	42

1 Introduction

Levshell is a context-aware research environment for the Sway window manager on Wayland-based Linux systems. Its purpose is to catalyze productivity, particularly for students and researchers in computer science and artificial intelligence. Unlike conventional desktop bars and panels, which display a fixed set of widgets regardless of user context, Levshell dynamically adapts its interface to the user’s current task, open projects, and cognitive state.

Levshell is more than a shell. It is designed as an **integrated research environment**: a suite of tools—knowledge base, reference manager, spaced repetition system, calendar, task manager—that share a unified data model and are orchestrated by a context-aware shell. The shell is the visible surface; the unified data model is the foundation.

1.1 Hybrid Architecture Strategy

Levshell adopts a **hybrid approach** to reach the goal of a fully integrated suite while remaining usable at every stage of development:

1. **Define the unified data model first.** Projects, notes, references, flashcards, events, tasks, and experiments are first-class entities in a shared schema (see Section 3.3). All shell logic—the context engine, ideation nudges, cross-tool queries—operates on this model, never on tool-specific data formats.
2. **Initially populate the model by syncing from existing tools.** Provider implementations for Zotero, Obsidian, Anki, and others act as *import/sync adapters* that translate external data into the unified model. The shell and daemon operate entirely on the unified model, not on the tools directly.
3. **Incrementally replace external tools with native components.** Over time, individual sync adapters are replaced by native Levshell components—a built-in knowledge base, a built-in SRS, a built-in reference manager—that write directly to the unified model. Each replacement is independent, and at every stage the system is fully functional.

This approach captures the core advantage of a full suite (unified data, deep cross-tool intelligence, total control over the user experience) while allowing the system to ship on top of existing tools immediately. The migration from “shell that syncs with Zotero” to “shell with built-in reference manager” is gradual, not a big bang.

Architectural Invariant

All shell logic depends on the unified data model, never on a specific tool or provider. Whether a reference entry was synced from Zotero or created natively, it is the same entity in the same schema. This invariant ensures that every native component replacement is transparent to the rest of the system.

1.2 System Components

The system is built on three foundational components:

1. **QuickShell (QML):** The rendering layer. Draws the bar, handles input, displays what the daemon tells it to.
2. **levshell-daemon (Rust):** The brain. Owns the unified data model, event bus, context engine, module system, and all integration logic.

3. **Unified Data Store:** A structured database (embedded, e.g., SQLite or sled) that holds the canonical representation of all research entities. Both sync adapters and native components read and write to this store.

1.3 Design Principles

Core Design Principles

1. **Unified model is the source of truth.** All shell logic operates on the shared data model. External tools are data sources, never direct dependencies of the shell’s behavior.
2. **Context-awareness over configuration.** The shell should infer what the user needs from observable signals (active window, workspace, time, running processes) and adapt automatically, with manual overrides always available.
3. **Quiet until important.** Widgets, notifications, and metrics should remain invisible during normal operation and surface only when their information becomes actionable.
4. **Elegance is not optional.** Every interaction, animation, and visual element must be deliberate. Visual noise is a productivity cost.
5. **Modularity at every layer.** Every feature is a module. Adding, removing, or reconfiguring a feature should never require touching the core system.
6. **The bar is the primary interface.** The top bar is not a passive display—it is the user’s command center, research dashboard, and ambient awareness layer.
7. **Researcher-native.** Features are designed around the workflows of academic researchers: reading papers, writing L^AT_EX, running experiments on remote clusters, managing citations, retaining knowledge.
8. **Incremental migration.** The system must be fully functional at every stage—from “sync adapter for every tool” to “fully native suite.” No big-bang migration.

2 Feature Specification

This section exhaustively enumerates the features planned for Levshell. Features are categorized by domain. Each feature may be implemented as a core component, a default module, or an optional extension.

2.1 Top Bar Layout & Global UX

2.1.1 Global Menu Bar

The active application’s menu hierarchy (File, Edit, View, etc.) renders in the top bar rather than inside the application window, following the macOS convention. This reclaims vertical space in every application and provides a single, predictable location for menu actions. For GTK and Qt applications that export menus over D-Bus (via the `com.canonical.AppMenu.Registrar` interface), menus are displayed natively. For applications without menu export, a fallback display shows the application name and focused window title.

2.1.2 Unified Command Palette

A single keystroke summons a search-driven overlay inspired by Spotlight, Raycast, and Rofi, but native to the shell. The command palette serves as the universal entry point for: application launching, shell command execution, file search, workspace switching, recent document access,

calculator and unit/currency conversion, Unicode/emoji lookup, and every Levshell subsystem (module settings, project search, Anki card creation, Zotero lookup, etc.). All searchable surfaces in the shell register themselves with the command palette as search providers.

2.1.3 Breadcrumb Workspace Indicator

Instead of numbered boxes, the workspace indicator displays a breadcrumb path: workspace name, separator, focused window title—elegantly truncated. Each segment is interactive: clicking navigates or renames; drag-and-drop moves windows between workspaces.

2.1.4 Morphing Bar Density

The bar operates in three density modes:

- **Full**: All widgets visible with labels.
- **Compact**: Icons only; hover to expand individual widgets.
- **Hidden**: Bar is invisible; auto-shows on pointer push to screen edge or keybind.

Transitions are animated. Density can be set per-workspace or globally, and overridden with a keybind.

2.1.5 Clock & Calendar Hub

The center clock doubles as a dropdown hub displaying: a mini calendar view, upcoming events (synced from CalDAV, Google Calendar, or local ICS files), a world-clock row for collaborators in other time zones, and a countdown timer for the next upcoming event that appears in the bar 15 minutes before start.

2.1.6 Notification Center

A single aggregation point for all desktop notifications (via the freedesktop notification specification). The dropdown groups notifications by application, supports inline reply for chat applications, and includes a focus mode toggle that silences all notifications except those from starred senders. Missed notifications persist and can be snoozed or pinned.

2.1.7 Quick-Settings Flyout

A single-click panel (analogous to macOS Control Center) providing: volume and brightness sliders, night-light warmth control, and toggles for Wi-Fi, Bluetooth, Do Not Disturb, screen recording, and VPN. Each tile can expand for detail (e.g., Wi-Fi shows available networks, Bluetooth shows paired devices with battery levels).

2.2 Productivity & Focus

2.2.1 Pomodoro / Session Timer

A compact pill in the bar showing elapsed focus time. Configurable work/break intervals with auto-logging of sessions per workspace. Break reminders respect Do Not Disturb state. Optionally dims or tints the screen during breaks.

2.2.2 Daily Planner Glance

A collapsible widget displaying today's agenda as a vertical timeline, sourced from calendar sync. Tasks from Todoist, Taskwarrior, or plain Markdown files can be interleaved. Tasks can be completed inline from the widget.

2.2.3 Scratch Pad / Quick Capture

A keybind opens a small floating text input anchored to the bar. Typed content is appended to a configurable capture file (Markdown, Org-mode, or Zettelkasten inbox). Supports image pasting, which saves images to an assets directory and inserts a link automatically.

2.2.4 Reading List Queue

A dropzone in the bar accepts dragged URLs or PDFs, adding them to a reading list. The widget displays an unread count and allows cycling through queued items.

2.2.5 Global Word/Character Count

When a text editor or word processor is focused, a subtle counter displays live word count, character count, and estimated page count.

2.2.6 Session Journal

On demand or at end-of-day, Levshell auto-generates a summary of the session: applications used, files touched, workspaces visited, focus time logged. Exportable as Markdown.

2.3 System Telemetry & Resource Awareness

2.3.1 Adaptive System Telemetry

CPU, RAM, temperature, disk I/O, and network throughput displayed as compact sparkline graphs. Widgets are configured to appear only when values exceed configurable thresholds (e.g., CPU > 60%), implementing the “quiet until important” principle.

2.3.2 Battery & Power Profile

Battery icon with estimated time remaining. Click to switch between power profiles (power-saver, balanced, performance) via `power-profiles-daemon` or TLP. Displays current wattage draw when hardware supports it.

2.3.3 Network Quality Indicator

Beyond connection status: displays latency to a configurable endpoint (e.g., university VPN gateway), throughput rates, and a color-coded quality dot (green/yellow/red).

2.3.4 Disk Space Warning

Invisible until free space on any mounted volume drops below a configurable threshold, at which point it surfaces with a cleanup shortcut.

2.3.5 Process Sniper

Long-press or right-click the CPU graph to see the top 5 resource-consuming processes. One click sends `SIGTERM`; shift-click sends `SIGKILL`.

2.4 Workspaces & Window Management

2.4.1 Named & Persistent Workspaces

Workspaces have human-readable names and optionally persist their window layout across reboots—which applications, which positions, which monitors.

2.4.2 Workspace Thumbnails

Hovering over a workspace indicator shows a live mini-preview of its contents, similar to macOS Mission Control.

2.4.3 Workspace Templates

Pre-defined layout templates that can be instantiated: e.g., “Research” opens a browser on the left half, PDF reader top-right, notes application bottom-right, all pre-tiled.

2.4.4 Window Grouping / Tabs

Multiple windows can be grouped into a tabbed container within a workspace. The tab bar is styled by Levshell and can reside in the top bar area or float above the group.

2.4.5 Scratchpad Manager

A bar widget displaying the number of windows in Sway’s scratchpad with a dropdown list for peeking and cycling through them.

2.4.6 Picture-in-Picture

Pin any window as a floating, always-on-top, resizable, opacity-adjustable thumbnail.

2.4.7 Workspace Activity Indicator

A subtle visual cue (glow or dot) on workspace labels that have had recent unacknowledged activity.

2.5 File & Clipboard Management

2.5.1 Clipboard History

A ring buffer of the last N clipboard entries (text, images, files). Searchable from the command palette or a dedicated dropdown. Supports pinning frequently-used snippets.

2.5.2 Clipboard Transformations

Paste-as actions: plain text, URL auto-linked, Markdown-to-HTML, formatted code block, or output of a user-defined transformation script.

2.5.3 Drag-and-Drop Shelf

A tray area in the bar for parking files or text snippets mid-workflow. Drag content to the shelf, switch workspaces, drag it out at the destination.

2.5.4 Recent Files

A dropdown of recently opened/modified files, grouped by application or workspace.

2.5.5 Quick File Preview

Hovering over a file in the shelf or recent-files list shows a thumbnail or text preview.

2.6 Search & Knowledge

2.6.1 Unified Desktop Search

Indexes local files (content and metadata), browser bookmarks, application entries, man pages, and notes. Results appear in the command palette with categorized sections. Custom search providers can be registered (Arch Wiki, arXiv, Stack Overflow, etc.).

2.6.2 Dictionary / Thesaurus Lookup

Highlight a word, invoke a keybind, and receive a popover with definition, synonyms, and etymology.

2.6.3 Quick Translation

Same interaction flow as dictionary lookup but sends the selection to a translation engine (DeepL, LibreTranslate, or a local model).

2.6.4 Formula Reference

A searchable sheet of mathematical formulas, Greek letters, and $\text{\LaTeX}$ symbol names. Selecting an entry copies the $\text{\LaTeX}$ command, Unicode character, or both.

2.6.5 Snippet / Template Library

A searchable repository of text templates—email boilerplate, $\text{\LaTeX}$ preambles, code snippets, regex patterns—insertable into the focused application.

2.7 Communication & Social

2.7.1 Unified Inbox Indicator

A single icon with a badge count aggregating unreads from email (IMAP/Notmuch), chat (Matrix, Slack, Discord), and calendar reminders. Clicking expands a grouped list.

2.7.2 Quick Reply

For supported protocols, reply inline from the notification dropdown.

2.7.3 Meeting Join Button

When a calendar event with a video-call link is approaching, a “Join” button appears in the bar.

2.7.4 Presence / Status Broadcast

Set status (“focusing,” “in class,” “available”) from the bar and broadcast to configured services simultaneously.

2.8 Media & Ambient

2.8.1 MPRIS Media Controls

Play/pause, track navigation, progress bar, and album art for the active MPRIS-compatible media player. Supports multiple simultaneous sources with a switcher.

2.8.2 Now-Playing Display

Current track title as scrolling text or tooltip. Optional scrobbling to Last.fm or ListenBrainz.

2.8.3 Per-Application Volume Mixer

Per-application volume control via PipeWire, accessible from a flyout.

2.8.4 Ambient Soundscapes

Built-in white noise, rain, or lo-fi generator (or controller for an external one) accessible from the bar.

2.8.5 Night-Light / Color Temperature

A warmth slider or automatic scheduling. Optional “paper” mode that shifts the color profile for extended reading.

2.9 Academic & Research

2.9.1 Open Project Registry

A first-class data model for “open projects.” Each project has a name, status (active, simmering, blocked, writing-up, complete), description, a list of open questions, associated tags, and links to external systems: Obsidian vault paths, Zotero collections, Git repositories, Anki tags, SSH hosts, BibTeX files, deadlines, and collaborators.

Projects are the fundamental unit of research context in Levshell. Other modules query the project registry to correlate their data with the user’s active research threads.

2.9.2 Stochastic Ideation Nudges

At semi-random intervals (Poisson-distributed around a configurable mean, e.g., $\lambda = 45$ minutes), a non-modal notification surfaces a prompt tied to an open project. Prompts are drawn from: the project’s open questions, recent Obsidian highlights, or detected cross-project connections. Nudges are suppressed during focus sessions and queued for delivery during transitions (workspace switch, screen unlock, break period).

2.9.3 Cross-Project Connection Surfacing

When a new Obsidian note or Zotero entry is added, the system checks for tag, keyword, and author overlaps with other open projects. Connections are surfaced as nudge-style notifications.

2.9.4 Project Pulse Indicator

A compact bar widget showing which projects have had activity today and which have been dormant beyond a configurable threshold. Clicking expands a dashboard: last touched, hours logged, upcoming deadlines, open questions, recent notes.

2.9.5 Blocked-Project Escalation

If a project is marked “blocked” or has been untouched for a configurable period, ideation nudge frequency increases and shifts to unblocking prompts: smallest concrete next step, or the user’s own recorded blocker note.

2.9.6 Anki Integration

- **Review due counter:** Persistent badge in the bar showing cards due. Color-coded by urgency. Click to open Anki or a minimal review overlay via AnkiConnect.
- **Inline card creation:** Highlight text anywhere, invoke keybind, populate a card creation popup. Auto-tags with the current workspace or project.
- **Context-triggered review:** When switching to a workspace associated with a topic that has due Anki cards, suggest reviewing those cards first.
- **Spaced repetition for open questions:** Open project questions are surfaced on a spaced repetition schedule, blurring the line between flashcards and the ideation engine.
- **Retention heatmap:** Dashboard view of retention rates by deck/tag over time.

2.9.7 Obsidian / Knowledge Base Integration

- **Graph-aware shell:** Indexes the vault’s link structure. Knows hubs, orphans, recent notes, and dense clusters.
- **Orphan note surfacing:** Periodically identifies notes with no backlinks and prompts the user to link or archive them.
- **Daily note integration:** Pulls today’s tasks and open loops from Obsidian daily notes into a bar widget. Inline completion syncs back to the Markdown file.
- **Vault search:** Full-text search across the vault from the command palette.
- **Context recall:** When returning to a workspace after absence, shows the last notes and files that were open, reducing re-entry cost.
- **Inbox monitor:** Badge for unprocessed fleeting notes with nudges to process them.
- **Note velocity:** Tracks notes created/modified per day/week, broken down by project.

2.9.8 Zotero & Literature Management

- **Library stats widget:** Total papers, unread count, recently added items.
- **Related work radar:** When a paper is added, checks its references and authors against the existing library for overlaps.
- **Reading progress:** Tracks current page in the active PDF reader per paper.
- **Annotation extractor:** Pulls highlights from PDFs and pushes them to Obsidian as literature note scaffolds.
- **Citation graph explorer:** Mini citation graph via Semantic Scholar or OpenAlex data.
- **Citation quick-search:** Bar widget to search BibTeX/Zotero and copy formatted citekeys.

- **Conference deadline integration:** Pulls deadlines from curated sources; escalates relevant project nudges as deadlines approach.
- **BibTeX health checker:** Periodic linting of `.bib` files for missing fields, duplicates, and unreferenced entries.

2.9.9 arXiv / Semantic Scholar Watcher

Configure keywords or followed authors. A badge appears when new matching papers are found. Click to see titles and abstracts; click through to open the PDF.

2.9.10 L^AT_EX Compilation Status

When a L^AT_EX build is running, a progress/status indicator appears in the bar (compiling → success/error). Click to view the log on error.

2.9.11 Lecture / Meeting Recorder

One-click screen and audio recording from the bar with automatic timestamping. Optional transcription via Whisper with transcript linked to notes.

2.9.12 Deadline Tracker

Upcoming deadlines from calendars, task managers, or plain text files. Color-coded by urgency.

2.10 SSH & Remote Systems

2.10.1 SSH Connection Dashboard

A bar widget showing all active SSH sessions: named hosts, connection duration, associated workspace, and a color-coded latency indicator (green/yellow/red). Immediate notification with one-click reconnect on connection drop.

2.10.2 Named Host Profiles

A host registry with human-readable names (“Lab GPU Cluster,” “University HPC”), associated project, and notes. These names replace raw hostnames throughout the shell.

2.10.3 Remote Job Monitor

For long-running remote jobs (training runs, benchmarks, compilations), the daemon polls status via log tailing over SSH, SLURM/PBS queue queries, or status files. A bar widget displays job name, host, progress, key metrics (e.g., epoch, loss), and ETA.

2.10.4 GPU Utilization Dashboard

Polls `nvidia-smi` or equivalent on remote hosts. Displays GPU utilization, memory usage, temperature, and running processes.

2.10.5 Remote Filesystem Bookmarks

Quick-access bookmarks to remote directories via SFTP/SSHFS. Click to mount; click again to open in a file manager or terminal.

2.10.6 SSH Tunnel Manager

Displays active port forwards and tunnels. Create new tunnels from the bar (e.g., forward local 8888 to remote 8888 for Jupyter). Visual indicator for active tunnels.

2.10.7 Experiment Tracker Integration

Surfaces run summaries from Weights & Biases, MLflow, or custom logging: which experiments ran, key metrics, running status.

2.10.8 Cluster Queue Status

Polls SLURM/PBS queues. Displays: system-wide queue depth, user's queued/running jobs, priority/fairshare, and estimated start time for pending jobs.

2.10.9 SSH Session Logging

Optional timestamped logging of SSH sessions (connection time, host, initial command) for correlation with the local session journal.

2.11 Code & Development Awareness

2.11.1 Build / CI Status

Polls GitHub Actions, GitLab CI, or local build systems. Displays red/green status per project. Click to view failure logs.

2.11.2 Git Status per Project

For each open project with a Git repository: current branch, uncommitted changes count, unpushed commits count, and divergence from upstream.

2.11.3 Compiler Error Flash

When a build fails in a terminal, captures the first error message and displays a persistent notification with file and line reference. Click to open the editor at that location.

2.11.4 Dependency Vulnerability Alerts

Periodic lightweight audits (`cargo audit`, `pip-audit`, `npm audit`) with badge notification on findings.

2.11.5 Docker / Container Status

Displays running containers, their exposed ports, and resource usage. One-click stop/restart.

2.11.6 Lab Notebook

Structured experiment log: hypothesis, setup (git commit, config, hardware), results, notes. Levshell auto-populates fields (commit hash, hostname, timestamp) on new entries.

2.12 Deep Focus & Cognitive Support

2.12.1 Warmup Mode

On session start (first unlock of the day or after long idle), offers a gentle ramp-up: review agenda, check Anki dues, glance at project statuses, then transition into focused work.

2.12.2 Context Save / Restore

A keybind snapshots the current state—open windows, terminal scrollback, editor positions, browser tabs, audio—and names it. Restoring returns to the exact state. This is the unit of context-switching.

2.12.3 Interruption Cost Awareness

When returning to a focused workspace after a context switch, a subtle timer indicates time since departure—not punitive, just awareness of re-entry cost.

2.12.4 Literature Review Mode

A context profile for deep reading: Zotero search prominent, annotation shortcuts visible, Anki card creation keybind active, media and notifications suppressed. Auto-triggered when a PDF reader is focused for a sustained period.

2.12.5 Writing Mode

When $\text{\LaTeX}$ or a Markdown editor is focused: word count visible, citation search prominent, build status shown, distraction-free bar option available.

2.12.6 Rubber Duck Debugger

A bar button opening a minimal chat interface (connected to a local LLM or structured prompt template) for articulating and debugging stuck points.

2.13 Data & Experiment Pipeline

2.13.1 Dataset Registry

A catalog of frequently-used datasets with metadata: location (local, remote, S3), size, last accessed, associated project.

2.13.2 Storage Quota Monitor

Monitors university storage, NFS mounts, and cloud buckets. Warns before quota limits are hit.

2.13.3 Reproducibility Snapshot

A keybind captures the current experimental environment: git hash, dependency freeze, hardware specs, config files. Written to a structured log.

2.13.4 Paper-to-Code Linker

Maintains mappings between Zotero entries and code repositories (both reference implementations and personal reimplementations).

2.14 Health & Wellbeing

Includes: eye break reminders (20-20-20 rule), posture/stretch reminders, daily screen time tracking (broken down by app/workspace), and an optional hydration tracker. All reminders are non-intrusive and respect focus sessions.

2.15 Security & Privacy

Includes: VPN status indicator with one-click connect, password manager integration (KeePassXC, Bitwarden CLI, **pass**) for OTP/password access from the bar, a visible screen lock button, and an always-visible camera/microphone activity indicator.

2.16 External Hardware & Peripherals

Includes: display layout manager with one-click presets (laptop only, docked, presentation), audio device switcher (PipeWire), Bluetooth device list with battery levels, and USB device mount/eject notifications.

2.17 AI & LLM Integration

2.17.1 Inline AI Assistant

A bar widget or command-palette mode that sends highlighted text or typed queries to a local LLM (Ollama, llama.cpp) or cloud API. Results appear in a popover. Use cases: summarization, rewriting, error explanation, regex deconstruction.

2.17.2 Smart Paste

Paste clipboard content through an LLM transformation: summarize, translate, convert to bullet points, reformat.

2.17.3 Contextual Suggestions

Based on workspace and active window, surface relevant information: related arXiv papers, Zotero entries matching searched terms, Anki cards due in the current topic.

2.18 Appearance & Theming

Includes: global color scheme synchronization (propagated to GTK, Qt, terminals, Sway borders), frosted-glass blur and transparency for the bar, smooth animations and micro-interactions, typography control (per-element font/weight/size), accent color per workspace, dark/light mode toggle with automatic scheduling, and a compact presentation mode that strips the bar to essentials and hides personal information.

2.19 Automation & Scripting

2.19.1 levshell-ctl: CLI Control Interface

A standalone binary (**levshell-ctl**) that connects to the daemon's Unix socket, sends a single command, prints the response, and exits. This enables external control of the shell from any context: Sway keybinds, shell scripts, cron jobs, or other tools. The command format follows a **target / action / params** convention:

```
levshell-ctl widget ssh-dashboard reconnect gpu-cluster-3
levshell-ctl palette query "zotero attention mechanisms"
levshell-ctl density compact
levshell-ctl profile activate writing
levshell-ctl project status llm-alignment
levshell-ctl anki due-count
levshell-ctl notify "Build finished" --urgency normal
```

This binary reuses the existing IPC message types (`ShellMessage` enum) and codec from `levshell-ipc`. It is a thin client: connect, serialize one message, read one response, print, exit. The daemon handles `levshell-ctl` connections alongside the primary QML shell connection by accepting multiple clients on the Unix socket. Integration with Sway keybinds is direct:

```
# In sway config
bindsym $mod+space exec levshell-ctl palette toggle
bindsym $mod+shift+n exec levshell-ctl density toggle
bindsym $mod+p exec levshell-ctl profile cycle
```

2.19.2 User-Defined Widgets

An API allowing user scripts (shell, Python, Lua) to output structured data that Levshell renders as bar widgets.

2.19.3 Trigger System

A lightweight event $\rightarrow$ action engine: “when workspace X becomes active, run command Y,” “when battery drops below 20%, switch to power-saver,” “when USB device is plugged in, mount and open.”

2.19.4 Keybind Cheat Sheet

A searchable overlay of all configured Sway and Levshell keybinds.

2.19.5 Shell Command Pinning

Pin the output of any shell command as a live-updating bar widget.

2.19.6 Extension / Plugin System

A distribution mechanism for community-packaged widgets, themes, and context profiles.

2.20 Meta-Productivity & Self-Awareness

Includes: research velocity dashboard (papers read, experiments run, commits pushed, writing output over time), weekly review prompt with auto-generated template, energy pattern observation (when deep work tends to happen, with scheduling suggestions), and procrastination pattern detection with gentle redirection nudges.

3 Technical Architecture

3.1 Core Components

Levshell comprises three foundational components:

Architectural Separation

levshell-daemon (Rust) owns the unified data model, event bus, context engine, sync engine, module system, and all integration logic. It is the brain.

Unified Data Store (SQLite) holds the canonical representation of all research entities—notes, references, flashcards, events, tasks, experiments, projects. Both sync adapters and native components read and write to this store. All shell logic operates on this model, never on external tool formats.

QuickShell (QML) renders the bar, handles mouse/keyboard input on its surfaces, and displays what the daemon tells it to. It is the face.

QML never talks to Sway, never talks to the data store, never polls anything. It receives a state description and renders it. It sends user interaction events back to the daemon.

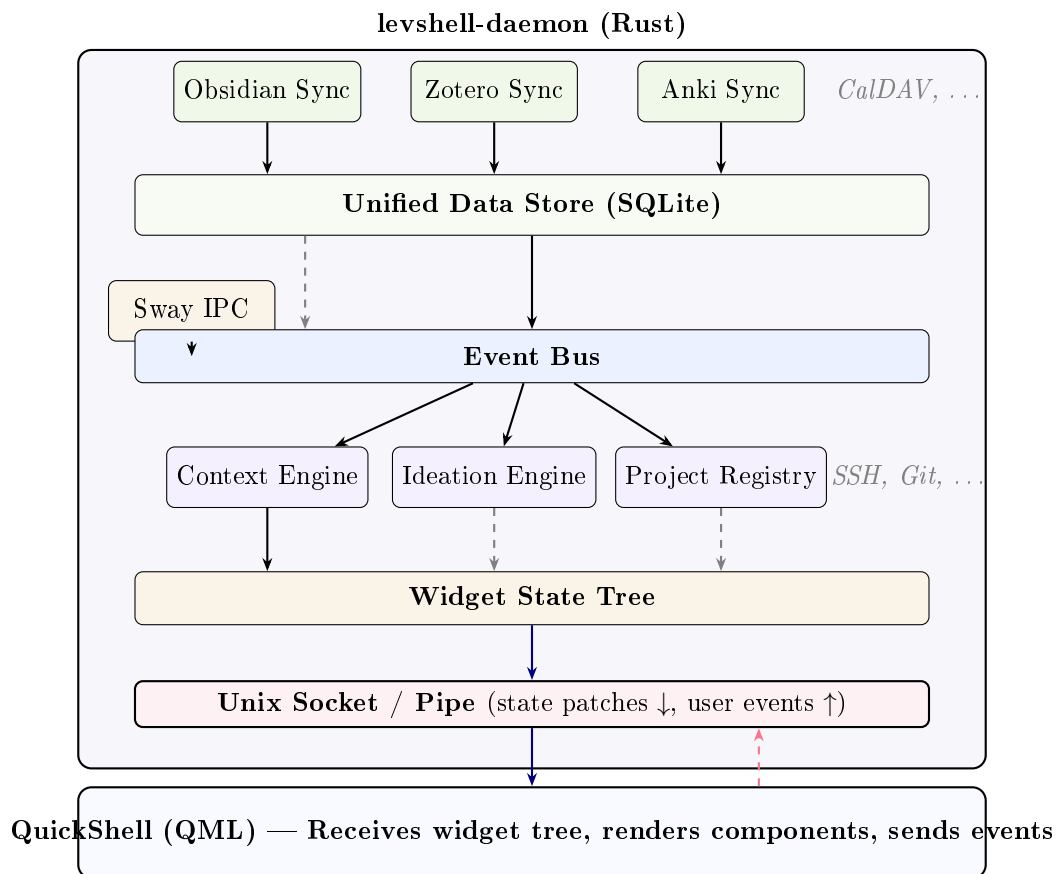


Figure 1: High-level architecture of Levshell. Sync adapters populate the unified data store; modules operate on the data store via the event bus. The widget state tree drives the QML rendering layer over IPC.

3.2 IPC Protocol

The IPC channel between the daemon and QuickShell must satisfy four requirements: low latency (widget updates feel instant), structured messages (no string parsing), bidirectional communication (daemon pushes state; QML pushes user events), and incremental updates (patches, not full state retransmission).

3.2.1 Transport

A Unix domain socket carrying NDJSON (newline-delimited JSON) messages. Each message is a single line of compact JSON terminated by `\n`. JSON's grammar forbids unescaped newlines inside strings, making `\n` a safe frame delimiter. This format was chosen over length-prefixed binary framing because QuickShell's `Io.Socket` API provides a `SplitParser` that splits on string delimiters but has no byte-level framing support. A per-line size cap of 16 MB guards against malformed frames that could stall the parser.

Messages are never pretty-printed on the wire (compact single-line JSON only). The QML side parses each line with `JSON.parse()`; the Rust side serializes with `serde_json::to_string()` and appends `\n`.

3.2.2 Codec Abstraction and Serialization Strategy

To allow migration to more efficient formats without protocol redesign, serialization is abstracted behind a codec interface:

```
trait Codec {
    fn encode(&self, msg: &Message) -> Result<Vec<u8>>;
    fn decode(&self, bytes: &[u8]) -> Result<Message>;
}
```

The v1 implementation uses `JsonCodec`. All message types are defined as Rust structs with `serde` derives, so switching to `MessagePack` is a single-line change (`serde_json::to_vec` → `rmp_serde::to_vec`). `FlatBuffers` requires schema files and a different access pattern but remains a viable future optimization.

Payload Overhead Analysis

In practice, most widget state changes are infrequent: workspace switches happen a few times per minute; SSH status updates every few seconds; Anki due counts change on review completion. High-frequency events (media progress, telemetry) produce at most 10–50 small messages per second. JSON parsing at this rate is microseconds of CPU work.

The dominant battery cost is **module polling overhead** (SSH latency checks, filesystem watchers, GPU monitoring), not IPC serialization. Accordingly, the daemon implements a global **power-aware mode**: when the system is on battery, all module poll intervals are automatically doubled (or set to a module-specific battery interval). This is configurable globally in `levshell.toml` and per-module in each module's configuration file.

3.2.3 Daemon → QML: State Patches

Each message is a targeted patch to the UI state. The daemon maintains a **widget state tree**—a tree of widget descriptors (type, properties, children, ordering, visibility). QML maintains a mirrored local state model and applies incoming patches reactively via QML property bindings. Example patch messages:

```
{ "op": "update", "path": "bar.left.ssh-dashboard",
```

```

"state": { "connections": [...], "total_latency_ms": 42 } }

{ "op": "reorder", "path": "bar.left",
  "children": ["workspace-indicator", "ssh-dashboard", "git-status"] }

{ "op": "hide", "path": "bar.right.media-controls" }

```

3.2.4 QML → Daemon: User Events

Structured event messages sent when the user interacts with a widget:

```

{ "event": "widget_action", "widget": "ssh-dashboard",
  "action": "reconnect", "data": { "host": "gpu-cluster-3" } }

{ "event": "command_palette", "query": "zotero attention mechanisms" }

{ "event": "density_change", "mode": "compact" }

```

3.3 Unified Data Model

The unified data model is the foundation of the hybrid architecture. It defines a shared schema for all research entities, independent of which external tool (if any) originally produced them. All shell logic—the context engine, ideation nudges, cross-tool queries, project tracking—operates exclusively on this model.

The data store is an embedded database within the daemon (SQLite for the initial implementation, chosen for its maturity, single-file simplicity, and excellent Rust support via `rusqlite`). The model is accessed by modules through a typed query API; modules never issue raw SQL.

3.3.1 Core Entity Types

```

Project {
  id: Uuid,
  name: String,
  status: Active | Simmering | Blocked | WritingUp | Complete,
  description: String,
  open_questions: Vec<OpenQuestion>,
  tags: Vec<String>,
  deadlines: Vec<Deadline>,
  collaborators: Vec<String>,
  accent_color: Option<String>,
  created_at: DateTime,
  updated_at: DateTime,
}

Note {
  id: Uuid,
  title: String,
  content: String,                // Markdown
  tags: Vec<String>,
  links: Vec<Uuid>,              // References to other Notes
  project_id: Option<Uuid>,
  source: SyncSource | Native,   // Provenance tracking
  created_at: DateTime,
  updated_at: DateTime,
}

```

```
Reference {
    id: Uuid,
    title: String,
    authors: Vec<String>,
    year: Option<u16>,
    venue: Option<String>,
    doi: Option<String>,
    citekey: String,
    abstract_text: Option<String>,
    tags: Vec<String>,
    collections: Vec<String>,
    pdf_path: Option<PathBuf>,
    reading_progress: Option<f32>,      // 0.0 to 1.0
    annotations: Vec<Annotation>,
    project_id: Option<Uuid>,
    source: SyncSource | Native,
    created_at: DateTime,
    updated_at: DateTime,
}

Flashcard {
    id: Uuid,
    front: String,
    back: String,
    tags: Vec<String>,
    linked_note: Option<Uuid>,
    linked_reference: Option<Uuid>,
    project_id: Option<Uuid>,
    // SRS scheduling state
    interval: Duration,
    ease_factor: f32,
    due_at: DateTime,
    review_count: u32,
    last_reviewed: Option<DateTime>,
    source: SyncSource | Native,
}

Event {
    id: Uuid,
    title: String,
    start: DateTime,
    end: DateTime,
    location: Option<String>,
    description: Option<String>,
    url: Option<String>,                // Video call link
    project_id: Option<Uuid>,
    recurrence: Option<RecurrenceRule>,
    reminders: Vec<ReminderOffset>,
    source: SyncSource | Native,
}

Task {
    id: Uuid,
    title: String,
    description: Option<String>,
    status: Pending | Active | Done | Cancelled,
    priority: Option<Priority>,
}
```

```

    due: Option<DateTime>,
    tags: Vec<String>,
    project_id: Option<Uuid>,
    source: SyncSource | Native,
}

Experiment {
  id: Uuid,
  name: String,
  project_id: Uuid,
  hypothesis: Option<String>,
  status: Queued | Running | Completed | Failed,
  host: Option<String>,           // SSH host where it ran
  git_hash: Option<String>,
  config: Option<serde_json::Value>,
  metrics: HashMap<String, f64>,
  started_at: Option<DateTime>,
  completed_at: Option<DateTime>,
  notes: Option<String>,
  source: SyncSource | Native,
}

```

3.3.2 Provenance Tracking

Every entity carries a **source** field that records its origin:

```

enum DataSource {
  Native,                               // Created within Levshell
  SyncSource {
    provider: String,                   // "zotero", "obsidian", "anki",
    ...,
    external_id: String,                // ID in the source system
    last_synced: DateTime,
    sync_direction: Bidirectional | ImportOnly | ExportOnly,
  },
}

```

This enables several critical behaviors: the sync engine knows which entities to push back to external tools (for bidirectional sync), the system can detect and resolve conflicts when both sides have been modified, and when a native component replaces a sync adapter, all existing entities remain valid—only their **source** field changes from **SyncSource** to **Native** once the external tool is no longer the authority.

3.3.3 Cross-Entity Relations

The data model supports typed links between entities:

- A **Note** can link to other **Notes** (knowledge graph), to **References** (literature notes), and to a **Project**.
- A **Flashcard** can link to the **Note** or **Reference** it was derived from.
- An **Experiment** links to its **Project** and optionally to **Notes** and **References**.
- A **Reference**'s annotations can be automatically scaffolded into **Notes**.
- A **Flashcard**'s review performance can be surfaced on its linked **Note** as a confidence signal.

These cross-entity relations are what make the unified model fundamentally more powerful than a collection of isolated tools. The query “show me everything related to project X” is a single indexed database query, not a multi-source aggregation job.

3.3.4 Sync Engine

The sync engine is a daemon subsystem that runs sync adapters on configurable intervals. Each adapter implements a common sync trait:

```
trait SyncAdapter {
  fn name(&self) -> &str;
  fn entity_types(&self) -> Vec<EntityType>; // What it syncs
  fn pull(&self, since: DateTime) -> Result<Vec<SyncDelta>>;
  fn push(&self, deltas: Vec<SyncDelta>) -> Result<()>;
  fn full_import(&self) -> Result<Vec<Entity>>;
  fn status(&self) -> SyncStatus; // Healthy, Stale, Error
}
```

Sync operates on deltas: on each pull, the adapter fetches entities modified since the last sync timestamp and upserts them into the unified store. Conflict resolution follows a last-write-wins strategy with optional manual resolution for detected conflicts. The sync engine emits events on the event bus (`SyncCompleted`, `SyncConflict`, `SyncError`) so modules can react.

Initial Sync Adapters

The v1 implementation targets sync adapters for: Zotero (SQLite database read + optional Zotero API for remote libraries), Obsidian (filesystem watcher on vault directory), AnkiConnect (HTTP JSON-RPC), and CalDAV (for calendar sync with Google Calendar, Nextcloud, etc.). Additional adapters for Taskwarrior, W&B, and SLURM are planned as subsequent additions.

3.3.5 Migration Path: Sync Adapter to Native Component

When a native component is ready to replace a sync adapter, the migration follows a defined sequence:

1. The native component is activated alongside the sync adapter in a read-only overlap period. Both populate the unified model; the native component is used for display and interaction, while the sync adapter continues to pull updates from the external tool.
2. The user migrates their workflow to the native component (e.g., creates new notes in Lev-shell’s knowledge base instead of Obsidian).
3. Once the user is satisfied, the sync adapter is deactivated. Existing entities in the unified model remain unchanged—they simply stop receiving updates from the external tool. The **source** field is optionally migrated from **SyncSource** to **Native**.
4. An export function remains available so data can be pushed back to the external tool’s format at any time (e.g., export the knowledge base as an Obsidian-compatible vault of Markdown files).

At every stage, the system is fully functional. No step requires downtime or data loss.

3.4 Module System

Every feature is implemented as a **module** within the daemon. A module is defined by a Rust trait (with a future path to out-of-process modules over IPC for extensibility in other languages).

3.4.1 Module Trait

A module provides the following interface:

```
Module {
    name()          -> &str                // Identity
    widgets()       -> Vec<WidgetDescriptor> // What it can render
    relevance()     -> Vec<RelevanceRule>    // When to show/hide
    config_schema() -> ConfigSchema          // Configurable parameters
    on_event(Event) // React to bus events
    tick()          // Periodic poll (optional)
    )
    start() / stop() // Lifecycle
}
```

3.4.2 Module Isolation

Modules do not reference each other directly. All inter-module communication flows through the **event bus**—a typed publish/subscribe system internal to the daemon.

3.4.3 Out-of-Process Modules

An external binary speaking the module protocol over a Unix socket or stdio. The daemon treats it identically to an in-process module from a protocol perspective. This enables modules written in Python, Go, or any language.

Security and Permission Model. Out-of-process modules running as native binaries on the user’s system have the same OS-level privileges as the user. This is consistent with every existing Linux desktop extension system (Polybar modules, GNOME Shell extensions, etc.) but must be mitigated through layered defenses:

1. **Tier 1 — Capability manifests.** Every out-of-process module ships a manifest declaring what it needs: filesystem paths, network access, specific IPC buses, environment variables. The daemon presents this manifest to the user at install time (analogous to mobile app permissions). This does not enforce restrictions at the OS level but creates a social contract and makes malicious intent visible through review.
2. **Tier 2 — Protocol-level restriction.** The module IPC protocol limits what a module can *ask the daemon to do*. A module can push widget state updates and subscribe to events—it cannot, through the protocol, read arbitrary files, access other modules’ internal state, or invoke arbitrary daemon functions. A compromised module can display misleading widgets and receive its subscribed events, but cannot exfiltrate data *through the daemon*.
3. **Tier 3 — WebAssembly sandboxing (recommended for community extensions).** For extensions from untrusted authors, the daemon offers a Wasm runtime (Wasmtime or Wasmer) as an alternative execution environment. Wasm modules run with *only* the capabilities the daemon explicitly grants via WASI—specific filesystem paths, specific network endpoints, nothing else. The tradeoff is reduced capability (no arbitrary system calls, no process spawning), but this is genuine sandboxing and is sufficient for most widget types.

4. **Tier 4 — OS-level confinement (optional hardening).** For native out-of-process modules, the daemon can optionally launch them under a restrictive AppArmor or Landlock profile that limits filesystem access, network access, and Linux capability usage. This requires system-level configuration but provides defense-in-depth for security-conscious users.

v1 Recommendation

Ship with Tier 1 (manifests) and Tier 2 (protocol restriction). Implement Tier 3 (Wasm) as the recommended and documented path for community extensions. Tier 4 (AppArmor/-Landlock) is a hardening option for future releases. The threat model should be documented honestly: native out-of-process modules are trusted code.

3.4.4 QML Widget Skins

The daemon specifies widget type and state; the QML side has a component file for each widget type. Users can override QML components to customize appearance without modifying the daemon.

3.5 Context Engine

The context engine is the subsystem that makes Levshell adaptive. It runs inside the daemon and is responsible for deciding which widgets are visible, where, and at what prominence level.

3.5.1 Input Signals

Signal	Source
Active workspace	Sway IPC subscription
Focused window (app_id, class, title)	Sway IPC subscription
Open windows per workspace	Sway IPC subscription
Time of day, day of week	System clock
Focus session state	Pomodoro / session timer module
Active context profile	Manual selection or auto-trigger rules
Associated project	Workspace configuration or inference
Module-specific conditions	Each module's internal state (e.g., “active SSH connections > 0”)

3.5.2 Resolution Layers

The context engine resolves widget layout through four layers, applied in order:

1. **Base layer.** A default widget layout that always applies: clock, workspace indicator, system tray, notification icon. This is the fallback when no context-specific rules match.
2. **Module relevance scores.** Each module declares relevance rules as predicates. Rules specify conditions under which a widget should be visible, compact, badge, expanded, or hidden. Conditions reference input signals and module-internal state.
3. **Context profiles.** Named bundles of relevance overrides. A “writing” profile might expand citation search, show word count, hide media controls, and suppress notifications. Profiles are activated manually, by keybind, or by auto-trigger rules.

4. **User override.** A keybind always toggles to the base layer or a “show everything” mode, guaranteeing that context-aware hiding never locks the user out of any widget.

3.5.3 Relevance Rule Format

Relevance rules are declared in module configuration files using a predicate DSL:

```
# Example: ssh-monitor module relevance
[relevance.ssh-dashboard]
default = "hidden"

[[relevance.ssh-dashboard.when]]
condition = "self.active_connections > 0"
prominence = "visible:compact"

[[relevance.ssh-dashboard.when]]
workspace_tag = "remote"
prominence = "expanded"
```

3.5.4 Conflict Resolution Algorithm

Widget prominence is resolved by a strict, deterministic cascade. Given the same input signals, the algorithm always produces the same output. The prominence levels form a total order: `hidden < badge < icon-only < compact < visible < expanded`.

1. **Step 1 — Base layout.** Every widget slot is assigned its default prominence from the base layer configuration. This produces a complete, valid layout with no gaps or conflicts.
2. **Step 2 — Module relevance rules.** For each widget, evaluate its owning module’s relevance predicates against the current context signals. If any predicate matches, its specified prominence *replaces* the base layer value. If multiple predicates within the same module match (e.g., both “active SSH connections > 0” and “workspace tagged remote”), the *most prominent* matching value wins. This is well-defined because prominence is a total order.
3. **Step 3 — Active context profile.** If a context profile is active (e.g., “writing mode”), its explicit overrides replace all previous values for the widgets it mentions. Profiles are user-authored and always take precedence over module self-assessment. Widgets not mentioned in the active profile retain their Step 2 values.
4. **Step 4 — Spatial constraint enforcement.** The bar has a finite pixel budget. Walk the widget list in *global priority order* (a user-configurable ordered list that ships with sensible defaults). Place each widget at its resolved prominence. If placing a widget at its current prominence would overflow the available space, demote it one level (e.g., `expanded` → `compact` → `icon-only` → `badge` → `hidden`). Highest-priority widgets receive first claim on bar real estate. This is a greedy $O(n)$ algorithm.
5. **Step 5 — User override.** If the user has explicitly pinned or hidden a widget via keybind or manual toggle, that decision is absolute and overrides all previous steps. A global “show everything” keybind sets all widgets to `visible`, bypassing the entire cascade.

3.5.5 Hysteresis and Flicker Prevention

To prevent visual instability when a condition oscillates around a threshold (e.g., CPU hovering at 59–61% around a 60% trigger), the context engine applies **hysteresis** to all condition-driven visibility changes:

- A condition must be continuously true for a minimum **activation delay** (default: 2 seconds) before it triggers a visibility change.
- Once activated, a condition must be continuously false for a longer **deactivation delay** (default: 10 seconds) before the change reverts.
- Both delays are configurable per-rule.

This ensures the bar layout is stable even when underlying signals are noisy. The computational cost is negligible: one timestamp comparison per active rule per evaluation cycle.

3.6 Event Bus

The event bus is the internal communication backbone of the daemon. Modules publish typed events and subscribe to events from other modules or from core subsystems.

3.6.1 Example Event Types

```
WorkspaceChanged      { name, focused_window }
SSHConnectionEstablished { host, pid, workspace }
SSHConnectionDropped { host, reason }
AnkiDueCountChanged   { total, by_tag: HashMap }
FocusSessionStarted   { project, duration }
FocusSessionEnded     { project, actual_duration }
IdleDetected          { duration }
FileModified          { path }
GitCommitPushed       { repo, branch, hash }
ExperimentStatusUpdate { id, host, metrics }
ZoteroItemAdded       { key, title, authors, tags }
ObsidianNoteModified  { path, title, tags, links }
NudgeDelivered        { project, question }
ProfileActivated       { name }
```

Any module can subscribe to any event type. The ideation engine subscribes broadly to maintain situational awareness. The context engine subscribes to events that affect widget layout. New modules subscribe only to the events they need, and publishing a new event type never requires changes to existing modules.

3.7 Project Registry

The project registry is a core daemon service that provides indexed access to **Project** entities in the unified data model (Section 3.3). It is not a module itself but a shared service that modules query. Projects are the fundamental unit of research context: they link notes, references, flashcards, experiments, tasks, and events into coherent research threads.

Projects can be created natively within Levshell or seeded from TOML configuration files under `~/.config/levshell/projects/`. Configuration-defined projects are loaded into the unified data store on startup and are hot-reloadable via `inotify`. The project registry also maintains runtime metadata not persisted in the data store: which workspaces are currently associated with each project, accumulated focus time for the current session, and the last-active timestamp.

In addition to the project entity fields defined in the unified data model, the registry tracks external system associations that are relevant to context inference but not part of the core data model:

```
ProjectConfig { // From TOML, merged with DB entity
```

```

git_repos: Vec<PathBuf>,
ssh_hosts: Vec<String>,
workspace_names: Vec<String>,
accent_color: Option<String>,
}

```

3.8 App-to-Context Mapping

The system maps running applications to contextual tags at three levels of user effort:

1. **Zero-effort defaults.** Levshell ships with a built-in mapping of common Wayland `app_id` values to context tags. For example: `org.pwmt.zathura` → `pdf-reader`; `com.obsidian` → `knowledge-base`; `Alacritty` → `terminal`.
2. **One-line configuration.** Users add custom mappings in a configuration file:

```

[apps.window-rules]
"org.texstudio" = { tags = ["latex", "writing"] }
"firefox" = { tags = ["browser"], context = "neutral" }
"slack" = { tags = ["communication"] }

```

3. **Workspace declarations.** Workspaces can be pre-declared with intent, so context is known before any window is inspected:

```

[workspaces.research-llm]
tags = ["research", "coding", "remote"]
project = "llm-alignment"
accent-color = "#7aa2f7"

```

3.9 Configuration Architecture

All configuration lives under `~/.config/levshell/` and is hot-reloadable via `inotify`. No daemon restart is required for configuration changes.

3.9.1 Directory Structure

```

~/.config/levshell/
levshell.toml           # Global: theme, bar density, keybinds
modules/
  ssh-monitor.toml      # SSH module config
  ideation.toml         # Nudge intervals, settings
  ...
sync/
  zotero.toml           # Zotero sync adapter config
  obsidian.toml         # Obsidian sync adapter config
  anki.toml             # AnkiConnect sync adapter config
  caldav.toml           # CalDAV sync adapter config
  ...
projects/
  llm-alignment.toml
  distributed-consensus.toml
  ...
profiles/
  writing.toml           # Context profile overrides
  debugging.toml

```

```

    reading.toml
    ...
rules/
  app-tags.toml          # Window class -> tag mappings
  auto-profiles.toml     # Condition -> profile triggers
themes/
  catppuccin-mocha.toml
  ...

~/.local/share/levshell/
levshell.db              # Unified data store (SQLite)
levshell.db-wal          # Write-ahead log
assets/                  # Attached images, PDFs, etc.

```

3.9.2 Configuration Precedence

Module defaults < module config file < context profile overrides < user keybind override. Each layer can only narrow or override; it cannot disable the user’s ability to manually override.

3.10 Extension Points

Extension Point	Description
Out-of-process modules	External binaries speaking the module protocol over Unix socket or stdio. Enables modules in any language.
QML widget skins	User-overridable QML component files for each widget type. Separates logic from presentation.
Scripting layer	Lua (via <code>mlua</code>) or Wasm for complex relevance rules and custom logic beyond static configuration.
Command palette providers	Any module or external script can register itself as a search provider in the command palette.
Trigger system actions	User-defined event → action rules, extensible with custom event types from out-of-process modules.
Theme packages	Distributable theme bundles containing color definitions, font specifications, and optional QML overrides.

4 Implementation Decisions

This section records the concrete technology choices and design decisions that govern the implementation. These are locked down prior to coding and should not be revisited without strong justification.

4.1 Rust Workspace Layout

The daemon is a Cargo workspace using a **layered crate split**. Crates are organized by architectural layer, not by feature. The dependency graph flows strictly downward—no cycles are permitted.

```

levshell/
Cargo.toml          # Workspace root
crates/

```

```

levshell-core/      # Event bus, module trait, module runner
,
# health state management
levshell-data/      # Unified data model, SQLite schema,
# query API, migrations, FTS5
levshell-ipc/       # Codec trait, message types,
# Unix socket server/client
levshell-context/   # Context engine, relevance rules,
# profiles, hysteresis
levshell-sync/      # SyncAdapter trait, sync engine,
# individual adapters (Zotero, Obsidian,
# ...)
levshell-config/    # Layered config loading, file watching,
# config struct definitions
levshell-modules/   # All built-in shell modules
levshell-daemon/    # Binary crate: main(), wires everything
,
# lifecycle management
levshell-ctl/       # CLI binary: one-shot commands to
daemon
# via Unix socket (connect, send, print,
# exit)
shell/              # QuickShell QML
  main.qml
  components/       # Shared QML components (degraded-state
# wrapper, animations, theme bindings)
  widgets/          # Per-widget-type QML components
config/             # Default and example configuration
  files
spec/               # This specification document

```

4.1.1 Crate Dependency Graph

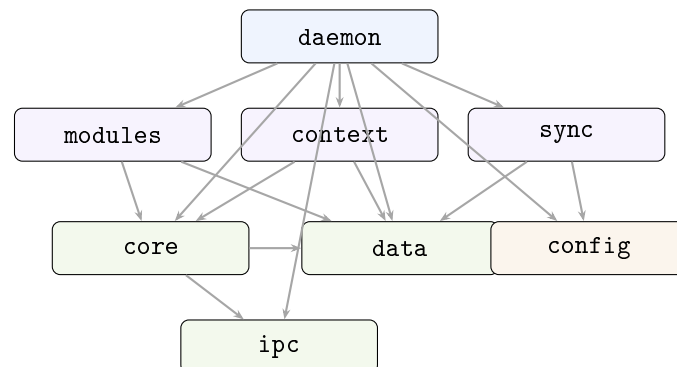


Figure 2: Crate dependency graph. All arrows point from dependent to dependency. The **daemon** crate (binary) depends on everything; leaf crates (**ipc**, **data**, **config**) have no internal dependencies.

4.1.2 Rationale

This split provides compile parallelism where it matters most: **levshell-data** and **levshell-ipc** change rarely once stable, so downstream crates avoid unnecessary recompilation during module iteration. The layered boundaries enforce the architectural invariant that modules cannot import sync adapter internals and the context engine cannot reach into module state. If

`levshell-modules` grows too large, individual modules can be extracted into their own crates without affecting other layers.

Sync adapters live as Rust modules within `levshell-sync`. The `SyncAdapter` trait is defined in `levshell-data` so that native components (in `levshell-modules`) can write to the data store without depending on `levshell-sync`.

4.2 Database and Schema Decisions

Decision	Choice and Rationale
Schema approach	Hybrid relational + JSON. Core queryable fields (title, status, project_id, citekey, due_at) are proper columns with indexes. Variable-shape data (annotations, experiment metrics, custom metadata) is stored as JSON in TEXT columns. This gives fast indexed queries on the hot paths while accommodating evolving data shapes without constant migrations.
Tags	Polymorphic join table. A single <code>entity_tags(entity_id BLOB, entity_type TEXT, tag TEXT)</code> table with indexes on (tag) and (entity_id, entity_type). Tag-based queries (“all entities tagged X,” tag overlap between projects) are fundamental to the ideation engine and context engine; the query penalty of JSON arrays is unacceptable.
Entity IDs	UUID v7, stored as BLOB(16). Globally unique (essential for sync adapters generating IDs before insertion), time-ordered (B-tree indexes maintain temporal locality for recent-data queries), and compact (16 bytes vs. 36 for TEXT representation). Generated via the Rust <code>uuid</code> crate with v7 support.
Relations	Explicit FKs + generic relation table. Primary, well-known relations (flashcard → note, experiment → project, note/reference/task/event → project) use nullable foreign key columns with indexes. The knowledge graph (note → note backlinks) and citation graph (reference → reference) use a generic <code>entity_relations(source_id, source_type, target_id, target_type, relation_kind)</code> table.
Provenance	Separate sync_metadata table. Entity tables carry only domain data. Sync provenance (provider name, external ID, last synced timestamp, sync direction, sync hash) lives in <code>sync_metadata(entity_id, entity_type, ...)</code> . Keeps entity tables clean; enables efficient sync-engine-wide queries.
Full-text search	FTS5 initially. FTS5 virtual tables shadow the <code>notes</code> and <code>references</code> tables, maintained via triggers. The query API abstracts over the search backend so Tantivy can be substituted later if needed.

Migrations	<code>rusqlite_migration</code> . Migrations are embedded as SQL string constants in the <code>levshell-data</code> crate, applied transactionally on daemon startup.
WAL mode	Enabled. <code>PRAGMA journal_mode=WAL</code> is set on database open for concurrent read/write performance.

4.3 Runtime and Library Decisions

Decision	Choice and Rationale
Async runtime	Tokio (multi-threaded, small thread pool of 2–4). Required by key dependencies: <code>swayipc</code> (Sway IPC), <code>request</code> (HTTP for AnkiConnect, W&B, arXiv), <code>tokio-tungstenite</code> (WebSocket, if needed). Fighting the ecosystem for an alternative runtime is not justified.
SQLite binding	<code>rusqlite</code> + <code>tokio::task::spawn_blocking</code> . <code>rusqlite</code> is the most mature Rust SQLite binding. SQLite is inherently synchronous; any “async” wrapper does the same <code>spawn_blocking</code> under the hood. The <code>levshell-data</code> crate exposes an async API externally while using synchronous <code>rusqlite</code> internally.
Event bus	Custom fan-out over <code>tokio::sync::mpsc</code> . Each subscriber gets its own bounded <code>mpsc</code> channel. The bus receives events and fans them out to subscriber channels, with per-subscriber filtering (modules only receive events they subscribed to). Slow consumers receive a log warning and dropped events rather than blocking the publisher. Events use a typed Rust enum (all types known at compile time for in-process modules). 50–100 lines of code atop <code>tokio::sync::mpsc</code> .
Configuration	Dedicated <code>levshell-config</code> crate using the <code>config</code> crate for layered configuration (files, environment variables, defaults) with <code>serde</code> deserialization. File watching via the <code>notify</code> crate for hot-reload. Config structs derive <code>serde::Deserialize</code> and are loaded from TOML files.
Error handling	thiserror in library crates, anyhow in the binary . Library crates define typed error enums (<code>DataError</code> , <code>SyncError</code> , <code>IpcError</code> , <code>ContextError</code>) enabling programmatic error matching. The <code>levshell-daemon</code> binary uses <code>anyhow</code> for top-level error propagation.
Serialization	<code>serde</code> + <code>serde_json</code> for IPC messages and JSON columns. The codec abstraction trait (Section 3.2) enables future migration to MessagePack (<code>rmp-serde</code>) without protocol changes.
UUID generation	<code>uuid</code> crate with v7 feature flag.

4.4 IPC Message Types (Phase 0)

The initial IPC protocol defines the minimum set of messages required for the first end-to-end module.

4.4.1 Daemon → QML

```
WidgetUpdate {
    widget_id: String,
    widget_type: String,
    state: serde_json::Value,      // Widget-specific, opaque to IPC
    status: WidgetStatus,          // Normal | Stale | Error |
        Unavailable
}

WidgetVisibility {
    widget_id: String,
    visible: bool,
    prominence: Prominence,        // Hidden | Badge | IconOnly |
        Compact                      //      | Visible | Expanded
}

BarLayout {
    left: Vec<String>,              // Ordered widget IDs
    center: Vec<String>,
    right: Vec<String>,
}
```

4.4.2 QML → Daemon

```
WidgetAction {
    widget_id: String,
    action: String,
    data: serde_json::Value,      // Action-specific payload
}

CommandPaletteQuery {
    query: String,
}

CommandPaletteSelect {
    provider: String,
    item_id: String,
}

DensityChange {
    mode: BarDensity,              // Full | Compact | Hidden
}
```

Using `serde_json::Value` for state and action payloads is deliberate: it keeps the IPC crate agnostic to specific widget state shapes. Each widget type defines its own state struct in the `modules` crate, serializes it for IPC, and the QML side parses it according to the widget type.

5 Architectural Constraints & Failure Modes

This section addresses known architectural risks and defines the strategies for mitigating them. These constraints are as fundamental to the design as the feature set itself.

5.1 Integration Brittleness and Sync Adapter Isolation

During the hybrid phase, Levshell relies on sync adapters to import data from third-party tools (Obsidian, Zotero, AnkiConnect, Taskwarrior, etc.) into the unified data model. Each of these tools may change its database schema, API surface, or file format between versions. Sync adapters are therefore the most maintenance-intensive and fragile layer of the system.

5.1.1 Design Constraint: Modules Never Touch External Tools

All shell modules operate exclusively on the unified data model (Section 3.3). No module ever queries Zotero’s SQLite database, parses Obsidian’s Markdown, or calls AnkiConnect’s HTTP API. Tool-specific code is confined entirely to sync adapter implementations, which are isolated, independently testable, and replaceable.

This means that when a sync adapter breaks due to an upstream change, the impact is contained: the affected entity types stop receiving updates (the sync engine emits **SyncError** events and the corresponding widgets transition to **Stale**), but all existing data in the unified model remains valid, all other modules and adapters continue operating normally, and the user can continue working with the last-known-good state while the adapter is repaired.

5.1.2 Sync Adapter Inventory

Entity Type	Initial Adapter	Alt. Adapters	Native Target
References	Zotero (SQLite API)	+ BibTeX file	Levshell Refs
Notes	Obsidian (filesystem)	Logseq, plain MD	Levshell KB
Flashcards	AnkiConnect (HTTP)	Mochi	Levshell SRS
Events	CalDAV	Google API	Levshell Cal
Tasks	Taskwarrior (CLI)	Todoist API	Levshell Tasks
Experiments	W&B (API)	MLflow	Levshell Lab
Jobs	SLURM (CLI)	PBS/Torque	—

The “Native Target” column indicates the planned native component that will eventually replace each sync adapter. Components marked “—” are expected to remain as sync adapters indefinitely, as they interface with external infrastructure (e.g., HPC clusters) rather than personal data stores.

5.1.3 Version Probing

On startup, each sync adapter performs a **version probe**: it checks the tool’s database schema version, API version, or binary version and logs a warning if the detected version is unrecognized. This provides early detection of incompatible upstream changes rather than silent data corruption or cryptic runtime errors.

5.1.4 Long-Term Brittleness Reduction

The hybrid architecture provides a natural path to reducing integration brittleness over time. As each native component matures and replaces its corresponding sync adapter, the number of external tool dependencies decreases. The end state—a fully native suite—has *zero* sync adapter brittleness. The migration sequence should prioritize replacing the adapters with the highest maintenance burden and most frequent upstream breakage first.

5.2 Widget Degraded State and Error Handling UI

When an external integration fails—a network request times out, a database query returns an error, a CLI tool is not installed—the user must never be left wondering whether the shell is frozen or working. Every widget supports four visual states, managed uniformly by the daemon’s module runner.

5.2.1 Widget Health States

State	Visual Treatment	Semantics
Normal	Default appearance	Data is fresh; the backing integration is healthy.
Stale	Data slightly dimmed; small clock icon	The last successful fetch was longer ago than expected. The widget displays last-known-good data but signals that it may be out-dated.
Error	Compact warning icon; tooltip with message	The module encountered an identifiable failure. Tooltip shows the error (“SLURM query timed out,” “AnkiConnect not running”). Click offers retry or settings link.
Unavailable	Widget hidden entirely	The module’s backing service is not configured or not installed. No visual noise for un-configured features.

5.2.2 Automatic State Transitions

The daemon’s module runner wraps every module’s external calls with a timeout and a staleness tracker. State transitions are managed by the runner, not by individual modules:

- If a module’s `tick()` or provider call has not successfully updated a widget’s state within $2\times$ the expected poll interval, the widget transitions to **Stale** automatically.
- If a call returns an explicit error, the widget transitions to **Error** with the error message attached.
- If the configured provider is absent or returns a “not installed” probe result, the widget is set to **Unavailable** at startup and excluded from layout calculations.
- A successful update always resets the state to **Normal**.

5.2.3 QML-Side Consistency

On the rendering side, every widget component receives a `status` field alongside its data payload: `"normal"`, `"stale"`, `"error"`, or `"unavailable"`. A shared QML wrapper component

implements the standard visual treatment for non-normal states, ensuring consistent degraded-state language across the entire bar. Individual widget components do not need to implement error handling themselves.

5.3 Power Awareness

The daemon implements a global **power-aware mode** that activates automatically when the system is running on battery (detected via `/sys/class/power_supply/` or `UPower`). Effects include:

- All module poll intervals are multiplied by a configurable factor (default: $2\times$).
- Modules may define a separate `battery_poll_interval` in their configuration that overrides the global multiplier.
- Animations are reduced or disabled.
- Non-essential background tasks (vault indexing, citation graph building, arXiv polling) are deferred until AC power is restored or explicitly requested.

The power state is exposed as an event on the event bus (`PowerStateChanged { on_battery: bool }`) so modules can implement additional power-saving behavior beyond poll interval adjustment.

6 Implementation Roadmap

This document captures the feature vision, hybrid architecture, implementation decisions, and architectural constraints. Implementation proceeds in phases, with each phase producing a working system. The implementation decisions in Section 4 are binding for all phases.

6.1 Phase 0: Foundation

The immediate work is to build the daemon core—the infrastructure that everything else depends on.

6.1.1 Step 0.1: Scaffold the Workspace

Create the Cargo workspace with all seven crates (Section 4.1). Each crate should compile (even if mostly empty) and the dependency graph should match Figure 2. Add `Cargo.toml` workspace members, set up shared dependency versions via workspace `[dependencies]`, and configure common settings (edition 2021). Create the `shell/` directory with a placeholder `main.qml`. Create the `config/` directory with example TOML files.

Key dependencies to add at this stage:

- `levshell-data`: `rusqlite` (with `bundled` feature), `rusqlite_migration`, `uuid` (with `v7` feature), `serde`, `serde_json`, `chrono`, `thiserror`, `tokio`
- `levshell-core`: `tokio`, `serde`, `thiserror`, `tracing`
- `levshell-ipc`: `tokio`, `serde`, `serde_json`, `thiserror`
- `levshell-config`: `config`, `serde`, `toml`, `notify`, `thiserror`
- `levshell-daemon`: `tokio` (full features), `anyhow`, `tracing`, `tracing-subscriber`

6.1.2 Step 0.2: Unified Data Model and Store

In `levshell-data`, implement:

1. The initial SQLite migration (Appendix A) containing all entity tables, the `entity_tags` join table, the `entity_relations` table, the `sync_metadata` table, and FTS5 virtual tables with triggers.
2. Rust struct definitions for all entity types (`Project`, `Note`, `Reference`, `Flashcard`, `Event`, `Task`, `Experiment`) with `serde` derives.
3. A `DataStore` struct that owns the SQLite connection (WAL mode), runs migrations on construction, and exposes an async query API wrapping `rusqlite` calls via `tokio::task::spawn_blocking`.
4. Basic CRUD operations: `insert`, `get_by_id`, `list` (with filtering and pagination), `update`, `delete` for `Project` and `Note` entities as the initial proof. Other entities follow the same pattern.
5. Tag operations: `add_tag`, `remove_tag`, `get_tags`, `find_by_tag`.
6. Full-text search: `search_notes(query)` and `search_references(query)`.

6.1.3 Step 0.3: Event Bus

In `levshell-core`, implement the typed event bus:

1. Define the `Event` enum with initial variants: `WorkspaceChanged`, `WindowFocused`, `DataStoreUpdated{entity_id, entity_id}`, `PowerStateChanged`.
2. Implement the `EventBus` struct: subscribers register with a set of event types they care about, each gets a bounded `mpsc::Receiver`. Publishing fans out to matching subscribers. Slow consumers get logged warnings and dropped events.
3. Define the `Module` trait (name, widgets, relevance rules, event handling, tick, lifecycle).
4. Implement the `ModuleRunner` that manages module lifecycles and wraps `tick()` calls with timeout and health state tracking.

6.1.4 Step 0.4: IPC Server

In `levshell-ipc`, implement:

1. Message type definitions: `DaemonMessage` (`WidgetUpdate`, `WidgetVisibility`, `BarLayout`) and `ShellMessage` (`WidgetAction`, `CommandPaletteQuery`, `CommandPaletteSelect`, `DensityChange`).
2. The `Codec` trait with a `JsonCodec` implementation.
3. An `IpServer` that listens on a Unix domain socket at a well-known path (e.g., `$XDG_RUNTIME_DIR/levshell`) accepts one connection (the QML shell), and provides async send/receive methods.

6.1.5 Step 0.5: Daemon Binary and First Module

In `levshell-daemon`, implement `main()`:

1. Initialize `tracing` for structured logging.
2. Open the `DataStore` (creates `~/.local/share/levshell/levshell.db`, runs migrations).

3. Start the `EventBus`.
4. Start the `IpcServer`.
5. Register and start the first module: a **Sway workspace module** (in `levshell-modules`) that subscribes to Sway IPC via the `swayipc-async` crate, emits `WorkspaceChanged` and `WindowFocused` events on the bus, and pushes `WidgetUpdate` messages for a workspace indicator widget.
6. In `shell/`, write a minimal QML bar that connects to the Unix socket, receives `WidgetUpdate` messages, and renders the workspace indicator. This validates the full pipeline end-to-end.

Phase 0 — Complete

Phase 0 shipped with 35 tests across the workspace (8 data + 10 core + 11 ipc + 5 indicator + 1 daemon end-to-end), zero clippy warnings. Delivered: Cargo workspace scaffold (8 crates), unified data model with SQLite schema and migrations, typed event bus with per-subscriber filtering, NDJSON IPC over Unix socket with codec abstraction, Sway workspace module, daemon binary with `ModuleFactory` pattern, and a minimal QuickShell QML bar. **Phase 0 deviations from original spec:** IPC framing was changed from 4-byte length-prefixed binary to NDJSON (newline-delimited JSON) because QuickShell 0.2.1's `Io.Socket` only supports `SplitParser` with string delimiters, not byte-level framing. The 16 MB per-line cap is retained. The `DataStoreUpdated` event uses `String` for `entity_type` (not `levshell_data::EntityType`) to keep `levshell-core` as a dependency-free leaf crate. Reference CRUD is minimal (insert + get only); other entity types have model structs but no repo methods yet. The data store uses a single `Mutex<Connection>` rather than a pool—adequate for Phase 0 write volumes.

6.2 Phase 1: Core Shell

With the foundation proven, build the core shell features: the context engine, essential bar widgets, the CLI control interface, and the visual language. Phase 1 turns the infrastructure into a usable daily-driver shell.

6.2.1 Step 1.1: `levshell-ctl` CLI

Create the `levshell-ctl` binary crate. This is a thin CLI client that connects to the daemon's Unix socket, sends a single command (serialized as a `ShellMessage`), reads one response, prints it, and exits. It reuses `levshell-ipc` message types and the NDJSON codec.

1. Add `crates/levshell-ctl/` to the workspace. Dependencies: `levshell-ipc`, `tokio` (minimal features), `clap` for argument parsing, `anyhow`.
2. Implement the `target / action / params` CLI grammar:

```
levshell-ctl widget <widget-id> <action> [params...]
levshell-ctl palette <toggle|query> [query-string]
levshell-ctl density <full|compact|hidden|toggle>
levshell-ctl profile <activate|cycle> [profile-name]
levshell-ctl status                                     # Daemon health summary
```

3. Modify `levshell-daemon` to accept multiple concurrent connections on the Unix socket: the primary QML shell connection (persistent, streaming) and zero or more `levshell-ctl` connections (ephemeral, one-shot). The daemon distinguishes them by behavior—a `ctl` con-

nection sends one message and the daemon responds then closes, while the shell connection remains open.

4. Add a **CtlResponse** message type to the daemon→client direction for one-shot command results (distinct from the streaming **DaemonMessage** types used for the QML shell).
5. Write integration tests: `levshell-ctl density compact` while the daemon is running, verify the density change event is emitted on the bus.

Sway Keybind Integration

Once `levshell-ctl` is functional, Sway keybinds can control Levshell directly:

```
# In ~/.config/sway/config
bindsym $mod+space exec levshell-ctl palette toggle
bindsym $mod+shift+n exec levshell-ctl density toggle
bindsym $mod+p exec levshell-ctl profile cycle
bindsym $mod+shift+s exec levshell-ctl widget ssh-dashboard
reconnect
```

This gives Levshell the same external scriptability as `DankMaterialShell`’s `dms ipc call` pattern, while keeping the persistent streaming socket for high-frequency QML updates.

6.2.2 Step 1.2: Context Engine

In `levshell-context`, implement:

1. The five-step conflict resolution algorithm (Section 3.5): base layout → module relevance → active profile → spatial constraints → user override.
2. The relevance rule predicate DSL, parsed from TOML configuration files. Rules reference context signals (workspace tags, focused app_id, module-internal conditions) and specify a target prominence level.
3. The hysteresis system: activation delay (default 2s), deactivation delay (default 10s), configurable per-rule.
4. Context profile loading from `~/.config/levshell/profiles/*.toml`.
5. Integration with the event bus: subscribe to **WorkspaceChanged**, **WindowFocused**, and **ProfileActivated** events; recompute widget layout on each relevant event; push **BarLayout** and **WidgetVisibility** messages over IPC.
6. Tests: verify deterministic resolution across all five steps, hysteresis prevents flicker on oscillating signals, profile overrides take precedence over module rules, spatial constraint demotion works correctly.

6.2.3 Step 1.3: System Telemetry Modules

Self-contained modules that exercise the module trait and widget rendering without needing sync adapters or the unified data model:

1. **CPU/RAM monitor.** Reads from `/proc/stat` and `/proc/meminfo`. Emits sparkline data. Implements “quiet until important” — widget is compact by default, expands when CPU > 60% or RAM > 80%.
2. **Battery module.** Reads from `/sys/class/power_supply/`. Emits **PowerStateChanged** events. Shows estimated time remaining, power profile switcher.

3. **Network quality.** Periodic latency probe to a configurable endpoint. Color-coded dot (green/yellow/red).
4. Each module declares relevance rules so the context engine can test real layout resolution with multiple competing widgets.

6.2.4 Step 1.4: Core Bar Widgets and Visual Language

Build the QML components that establish the shell’s visual identity:

1. **Clock and calendar hub.** Center clock that drops down to show a mini calendar, upcoming events placeholder (will be populated when CalDAV sync adapter lands in Phase 2), and world-clock row.
2. **Quick-settings flyout.** Volume, brightness, night-light, Wi-Fi toggle, Bluetooth toggle, Do Not Disturb. Each tile backed by a daemon module or a `levshell-ctl` action.
3. **Notification center.** Implement the freedesktop notification spec listener (via D-Bus) as a daemon module. QML renders grouped notifications with inline reply support for supported apps.
4. **Morphing bar density.** Implement the three density modes (full, compact, hidden) with animated transitions, controllable via `levshell-ctl density` and keybind.
5. **Degraded-state wrapper.** The shared QML component that applies visual treatment (dimming, warning icon, tooltip) for widgets in Stale or Error states. Every widget uses this wrapper.

6.2.5 Step 1.5: Command Palette

1. A search-driven overlay triggered by keybind (via `levshell-ctl palette toggle`).
2. Search provider interface: each provider registers a name and a query handler. Initial providers: application launcher (reading `.desktop` files), workspace switcher, and unified data model search (notes and references via FTS5).
3. Results are categorized, ranked, and rendered in sections. Selecting a result dispatches to the provider’s action handler.

Phase 1 Completion Criteria

Phase 1 is complete when: (1) `levshell-ctl` can send commands to the daemon and receive responses; (2) the context engine resolves widget layout deterministically from workspace/window signals, module relevance, and profiles; (3) at least three system telemetry modules render live data in the bar; (4) the notification center captures and displays desktop notifications; (5) the command palette launches applications and searches the data model; (6) the bar supports all three density modes with animated transitions. At this point, Levshell is a functional daily-driver shell (without research-specific features).

6.3 Phase 2: Sync Adapters and Research Features

Bring research data into the unified model and build the modules that operate on it.

1. **Sync adapter framework.** Implement the `SyncAdapter` trait, the sync engine scheduler, conflict detection, and the `SyncCompleted`/`SyncError` event types.

2. **First sync adapters.** Obsidian (filesystem watcher), Zotero (SQLite reader), AnkiConnect (HTTP). Each adapter populates the unified model; corresponding modules render widgets from model queries.
3. **Project registry.** Load projects from TOML config, link them to entities in the unified model, and expose them to the context engine and ideation engine.
4. **SSH monitor, remote job monitor, GPU dashboard.** High-value modules for the CS/AI research workflow.
5. **Ideation engine.** Stochastic nudges, cross-project connection surfacing, blocked-project escalation.

6.4 Phase 3: Native Components (Ongoing)

Begin replacing sync adapters with native Levshell components, prioritized by maintenance burden and integration depth gain.

1. **Levshell SRS.** Replace the AnkiConnect sync adapter with a native spaced repetition system. First candidate: AnkiConnect is fragile, the SRS algorithm is well-understood (SM-2 or FRS), and native SRS enables deep integration.
2. **Levshell Knowledge Base.** Replace the Obsidian sync adapter with a native Markdown knowledge base. Enables native graph-aware features, inline card creation, and a unified editing experience.
3. **Levshell Calendar, Task Manager, Reference Manager.** Subsequent native components, each following the migration path defined in Section 3.3.

6.5 Deferred

- Security model for out-of-process extensions (Tier 3 Wasm sandboxing).
- Extension/plugin distribution system.
- Global menu bar (D-Bus AppMenu integration).
- AI/LLM integration.

A Initial SQLite Schema

This is the complete initial migration applied by `levshell-data` on first startup. It should be embedded as a string constant in the `levshell-data` crate and applied via `rusqlite_migration`. The schema is presented in three parts for readability: entity tables, cross-cutting tables (tags, relations, sync metadata), and full-text search with triggers.

Entity Tables

```
-- =====
-- Levshell Unified Data Model - Migration 001
-- =====

PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;

-- -----
-- Projects
-- -----
CREATE TABLE projects (
  id          BLOB(16) PRIMARY KEY,    -- UUID v7
  name        TEXT NOT NULL,
  status       TEXT NOT NULL DEFAULT 'active'
              CHECK (status IN ('active','simmering',
                                'blocked','writing_up','complete')),
  description  TEXT NOT NULL DEFAULT '',
  open_questions TEXT NOT NULL DEFAULT '[]', -- JSON array
  created_at   TEXT NOT NULL,           -- ISO 8601
  updated_at   TEXT NOT NULL
);

-- -----
-- Notes
-- -----
CREATE TABLE notes (
  id          BLOB(16) PRIMARY KEY,
  title        TEXT NOT NULL,
  content      TEXT NOT NULL DEFAULT '',    -- Markdown
  project_id   BLOB(16) REFERENCES projects(id)
              ON DELETE SET NULL,
  created_at   TEXT NOT NULL,
  updated_at   TEXT NOT NULL
);

CREATE INDEX idx_notes_project ON notes(project_id);

-- -----
-- References (literature)
-- -----
CREATE TABLE refs (
  id          BLOB(16) PRIMARY KEY,
  title        TEXT NOT NULL,
  authors      TEXT NOT NULL DEFAULT '[]', -- JSON array
  year         INTEGER,
  venue        TEXT,
  doi          TEXT,
  citekey      TEXT NOT NULL,
  abstract_text TEXT,
  pdf_path     TEXT,
  reading_progress REAL DEFAULT 0.0,
```

```

    annotations    TEXT NOT NULL DEFAULT '[]', -- JSON array
    project_id     BLOB(16) REFERENCES projects(id)
                  ON DELETE SET NULL,
    created_at     TEXT NOT NULL,
    updated_at     TEXT NOT NULL
);

CREATE UNIQUE INDEX idx_refs_citekey ON refs(citekey);
CREATE INDEX idx_refs_project ON refs(project_id);

-- -----
-- Flashcards
-- -----
CREATE TABLE flashcards (
    id              BLOB(16) PRIMARY KEY,
    front           TEXT NOT NULL,
    back            TEXT NOT NULL,
    linked_note_id  BLOB(16) REFERENCES notes(id)
                  ON DELETE SET NULL,
    linked_ref_id   BLOB(16) REFERENCES refs(id)
                  ON DELETE SET NULL,
    project_id      BLOB(16) REFERENCES projects(id)
                  ON DELETE SET NULL,
    -- SRS scheduling state
    interval_days   REAL NOT NULL DEFAULT 1.0,
    ease_factor     REAL NOT NULL DEFAULT 2.5,
    due_at          TEXT NOT NULL,
    review_count    INTEGER NOT NULL DEFAULT 0,
    last_reviewed   TEXT,
    created_at      TEXT NOT NULL,
    updated_at      TEXT NOT NULL
);

CREATE INDEX idx_flashcards_due
    ON flashcards(due_at);
CREATE INDEX idx_flashcards_project
    ON flashcards(project_id);

-- -----
-- Events (calendar)
-- -----
CREATE TABLE events (
    id              BLOB(16) PRIMARY KEY,
    title           TEXT NOT NULL,
    start_at        TEXT NOT NULL,
    end_at          TEXT NOT NULL,
    location         TEXT,
    description      TEXT,
    url             TEXT,
    project_id      BLOB(16) REFERENCES projects(id)
                  ON DELETE SET NULL,
    recurrence       TEXT, -- iCal RRULE or JSON
    reminders        TEXT NOT NULL DEFAULT '[]',
    created_at      TEXT NOT NULL,
    updated_at      TEXT NOT NULL
);

CREATE INDEX idx_events_start ON events(start_at);
CREATE INDEX idx_events_project ON events(project_id);

-- -----
-- Tasks
-- -----

```

```
CREATE TABLE tasks (
  id          BLOB(16) PRIMARY KEY,
  title       TEXT NOT NULL,
  description  TEXT,
  status      TEXT NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending','active',
                              'done','cancelled')),
  priority    TEXT CHECK (priority IN ('low','medium',
                                      'high','urgent')),
  due_at      TEXT,
  project_id  BLOB(16) REFERENCES projects(id)
              ON DELETE SET NULL,
  created_at  TEXT NOT NULL,
  updated_at  TEXT NOT NULL
);
```

```
CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_due ON tasks(due_at);
CREATE INDEX idx_tasks_project ON tasks(project_id);
```

```
-- -----
-- Experiments
-- -----
CREATE TABLE experiments (
  id          BLOB(16) PRIMARY KEY,
  name        TEXT NOT NULL,
  project_id  BLOB(16) NOT NULL
              REFERENCES projects(id)
              ON DELETE CASCADE,
  hypothesis  TEXT,
  status      TEXT NOT NULL DEFAULT 'queued'
              CHECK (status IN ('queued','running',
                              'completed','failed')),
  host        TEXT,
  git_hash    TEXT,
  config      TEXT,
  metrics     TEXT NOT NULL DEFAULT '{}',
  notes       TEXT,
  started_at  TEXT,
  completed_at TEXT,
  created_at  TEXT NOT NULL,
  updated_at  TEXT NOT NULL
);
```

```
CREATE INDEX idx_experiments_project
  ON experiments(project_id);
CREATE INDEX idx_experiments_status
  ON experiments(status);
```

Cross-Cutting Tables

```
-- -----
-- Tags (polymorphic join table)
-- -----
CREATE TABLE entity_tags (
  entity_id   BLOB(16) NOT NULL,
  entity_type TEXT NOT NULL
              CHECK (entity_type IN ('project','note',
                                    'ref','flashcard','event',
                                    'task','experiment')),
  tag         TEXT NOT NULL,
  PRIMARY KEY (entity_id, entity_type, tag)
```

```

);

CREATE INDEX idx_tags_by_tag ON entity_tags(tag);
CREATE INDEX idx_tags_by_entity
  ON entity_tags(entity_id, entity_type);

-----
-- Entity relations (knowledge/citation graph)
-----
CREATE TABLE entity_relations (
  source_id      BLOB(16) NOT NULL,
  source_type    TEXT NOT NULL,
  target_id      BLOB(16) NOT NULL,
  target_type    TEXT NOT NULL,
  relation_kind  TEXT NOT NULL,
  created_at     TEXT NOT NULL,
  PRIMARY KEY (source_id, source_type,
               target_id, target_type,
               relation_kind)
);

CREATE INDEX idx_relations_target
  ON entity_relations(target_id, target_type);

-----
-- Sync metadata
-----
CREATE TABLE sync_metadata (
  entity_id      BLOB(16) NOT NULL,
  entity_type    TEXT NOT NULL,
  provider       TEXT NOT NULL,
  external_id    TEXT NOT NULL,
  last_synced_at TEXT NOT NULL,
  sync_direction TEXT NOT NULL
  DEFAULT 'import_only'
  CHECK (sync_direction IN (
    'bidirectional',
    'import_only',
    'export_only')),
  sync_hash      TEXT,
  PRIMARY KEY (entity_id, entity_type, provider)
);

```

Full-Text Search and Triggers

```

-----
-- Full-text search (FTS5)
-----
CREATE VIRTUAL TABLE notes_fts USING fts5(
  title, content,
  content=notes, content_rowid=rowid
);

CREATE VIRTUAL TABLE refs_fts USING fts5(
  title, abstract_text, citekey,
  content=refs, content_rowid=rowid
);

-- Triggers: keep FTS in sync with source tables

CREATE TRIGGER notes_ai AFTER INSERT ON notes BEGIN
  INSERT INTO notes_fts(rowid, title, content)

```

```
VALUES (new.rowid, new.title, new.content);
END;

CREATE TRIGGER notes_ad AFTER DELETE ON notes BEGIN
  INSERT INTO notes_fts(notes_fts, rowid, title, content)
  VALUES ('delete', old.rowid, old.title, old.content);
END;

CREATE TRIGGER notes_au AFTER UPDATE ON notes BEGIN
  INSERT INTO notes_fts(notes_fts, rowid, title, content)
  VALUES ('delete', old.rowid, old.title, old.content);
  INSERT INTO notes_fts(rowid, title, content)
  VALUES (new.rowid, new.title, new.content);
END;

CREATE TRIGGER refs_ai AFTER INSERT ON refs BEGIN
  INSERT INTO refs_fts(rowid, title,
                      abstract_text, citekey)
  VALUES (new.rowid, new.title,
          new.abstract_text, new.citekey);
END;

CREATE TRIGGER refs_ad AFTER DELETE ON refs BEGIN
  INSERT INTO refs_fts(refs_fts, rowid, title,
                      abstract_text, citekey)
  VALUES ('delete', old.rowid, old.title,
          old.abstract_text, old.citekey);
END;

CREATE TRIGGER refs_au AFTER UPDATE ON refs BEGIN
  INSERT INTO refs_fts(refs_fts, rowid, title,
                      abstract_text, citekey)
  VALUES ('delete', old.rowid, old.title,
          old.abstract_text, old.citekey);
  INSERT INTO refs_fts(rowid, title,
                      abstract_text, citekey)
  VALUES (new.rowid, new.title,
          new.abstract_text, new.citekey);
END;
```